IT159: Artificial Intelligence
Lab#2/Assignment#2: Search

1. Aim

The aim of this lab is for you to understand how different search strategies work, and to gain experience with a domain-specific application of search algorithms. You will explore the strengths and weaknesses of different search strategies and acquire deeper hands-on knowledge to complement your knowledge about search theory.

2. About the lab

This lab consists of two parts, one practical and one theoretical. The first part is a programming assignment where you are to implement search algorithms for an agent in a vacuum cleaner domain. You will implement search algorithms to make this work. The second part of the lab is to study and compare different search algorithms. The analysis should include the time and space complexity of the algorithms, and whether they are optimal or complete. To help you in the analysis, there are a number of questions for you to answer.

3. Lab system

The lab system consists of an interactive vacuum cleaner domain simulator with a simplified graphical representation suitable for debugging search algorithms. In this lab session, several different search algorithms can be used and we encourage experimentation! Your task is to implement your own version of breadth-first search and depth-first search according to the directions below.

The environment simulator

You can test your agent in *the simulator GUI* program. The simulator loads an agent (yours will be implemented in `myvacuumagent.py`) and executes it in a generated environment. Agents execute actions in the environment depending on what they can observe (perceive). The actions influence the environment, which in turn generates new sensory impressions for the agents. The simulator handles this interaction in a discrete manner; the world advances in steps, where each step can be described as follows:

1. The agent perceives the world
2. The agent decides on an action
3. The action is executed

The agent should be autonomous; therefore, the simulator makes no assumptions about the agent except that it can produce a reaction based on the sensory inputs presented to it. The agents make no assumptions about the simulator's implementation either. This disengagement is important, as

it enables comparison and evaluation of different agents' behaviours. Note that an agent has no authorisation to change the world directly, for example, via some global reassignment. This disengagement keeps the code modular and independent, allowing the representation of the world to change while keeping the same agent.

The implementation of the simulator

The simulator supplied for this lab is based on the AIMA-Java/AIMA-Python framework, but it is not identical to the original version available on the web.

The interface

Basically, the agents may perceive dirt or obstacles in the squares, and they can act by moving between the squares and removing dirt.

Percepts

A percept for the vacuum-agent consists of three values:

1. bump equal to true if the agent has bumped into an obstacle, else false.
2. dirt equal to true if the current square contains dirt, else false.
3. home equal to true if the agent is at the start position, else false.

The three Boolean values are obtained with the following code located in the execute method of an agent:

```
bump = percept.attributes["bump"]
dirt = percept.attributes["dirt"]
home = percept.attributes["home"]
```

Note that this is the only data that the agent gets from the simulator.

Output

The output from the agent's program (return value) is one of the following actions:

1. ACTION_FORWARD moves the agent one step forward depending on its direction,
2. ACTION_TURN_LEFT makes the agent turn left.
3. ACTION_TURN_RIGHT makes the agent turn right.
4. ACTION_SUCK cleans the dirt from the agent's current location.
5. ACTION_NOP informs the simulator that the agent has finished cleaning.

GUI

The provided environment simulator application is presented in Figure 2.

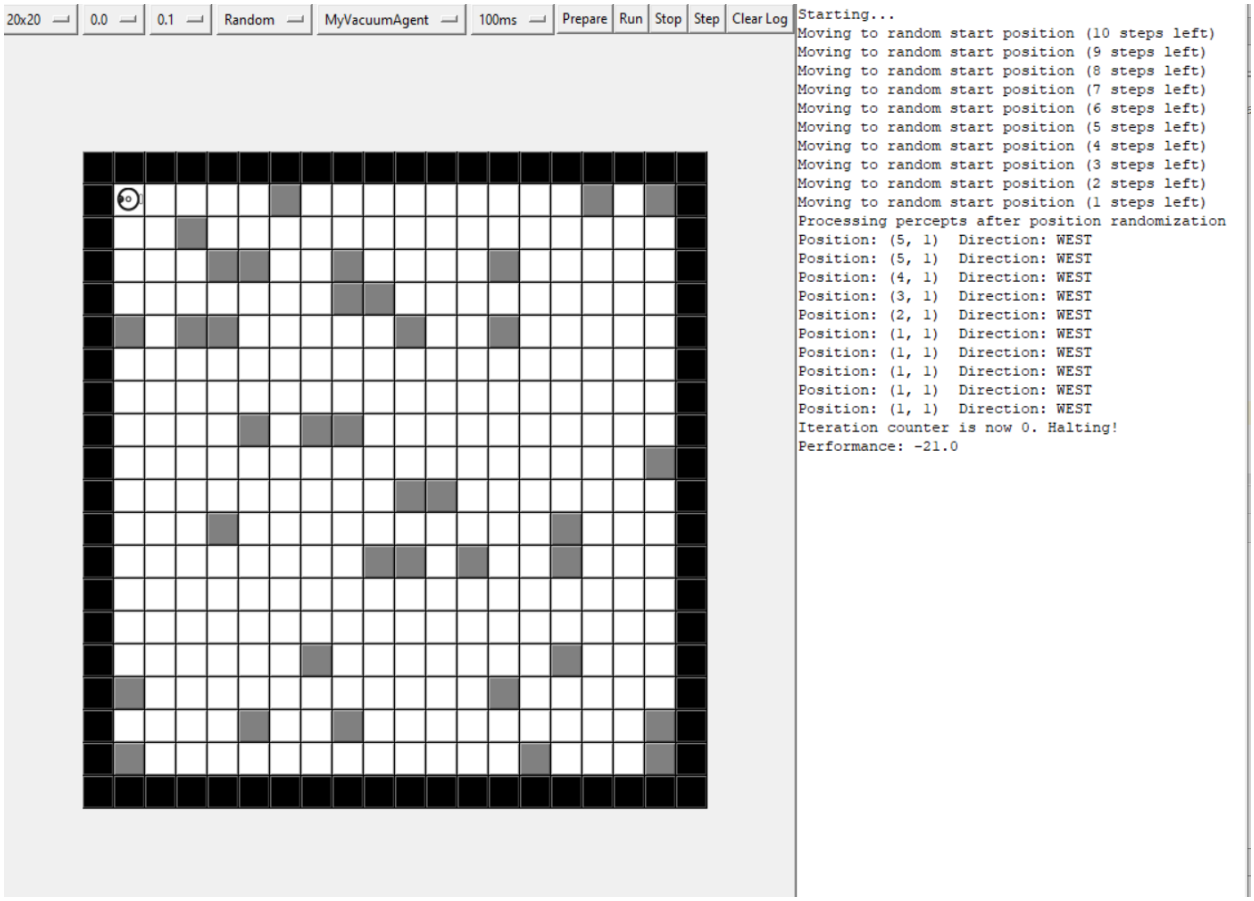

Figure 1: *Example of an environment in the vacuum agent's world*

The main window of the application is divided into two parts: the environment visualisation pane on the left and the text output pane, which displays the redirected *self.log* in Python. Certain messages from the simulator are printed in this pane, but it is also useful to debug your agent's code.

In the environment pane, the black squares represent obstacles. Trying to move an agent into an obstacle results in perceiving the bump equal to true. The grey squares represent the dirt. Moving into such a square results in perceiving the dirt equal to true. The graphical representation of the agent visualises its current direction.

The following drop-down menus are available to set the simulation parameters (from left):

1. *Environment size* can be set to *5x5, 10x10, 15x15, 20x20, 5x10, and 10x5*. Your agent should not be tailored to any specific size, though for convenience, you can assume a maximum size of *20x20*.
2. *Hinder probability* indirectly specifies the number of obstacles in the environment (it should be set to 0.0 and 0.1, in Tasks 1 and 2, **respectively**).
3. *Dirt probability* indirectly specifies the amount of dirt in the environment.

4. *Random environment* specifies whether a new environment should be generated for every run. This function is useful for testing an agent in the same environment multiple times.
5. *Type of agent* lists the available agent type classes. The first on the list is the agent you will modify.
6. *Delay* specifies the delay between each simulation step.

The following buttons are available (from left):

1. *Clear* cleans the text output pane.
2. *Prepare* generates a random environment based on the selected parameters.
3. *Run* starts the simulation.
4. *Step* allows for performing one step of the simulation at a time. Make sure to execute Prepare first.
5. *Cancel* terminates the simulation.

Program

The interesting part of the agent is the execute method. It is a function that takes a percept as input and returns an action as output. The simulator computes what the agent can perceive (a percept), applies the function (your program) to the percept, and executes the resulting action. Note (again) that the agent has no authorisation to change the world directly, for example, via some global reassignment.

As you can see from the definitions in the source code, the *RandomVacuumAgent* and *ReactiveVacuumAgent*, do not remember what they have done and experienced. If we want the agent to have a memory, we have to store information "inside" the agent.

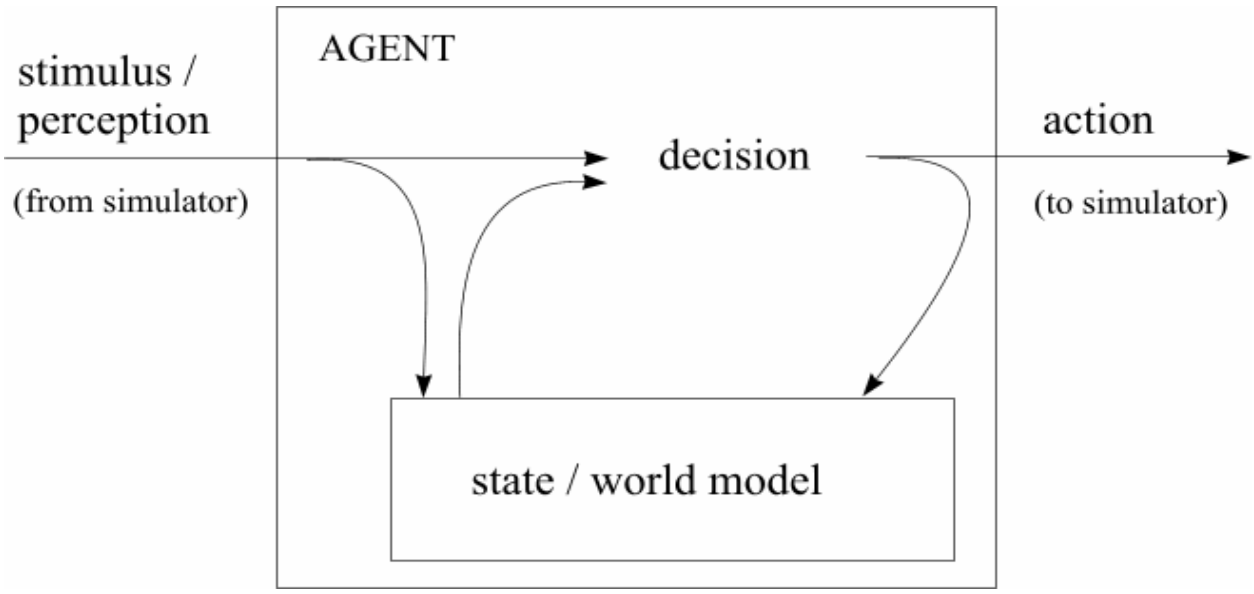

Figure 3: Agent with internal state

The state can be a model of the surrounding world as well as some other knowledge that the agent has, e.g. plans.

Scoring

The performance of an agent is measured by a score. The agent's initial score is -1000, and the score changes with each action. Sucking up dirt increases the score by 100 points, while shutting down at the starting square gives it 1000 points. All other actions decrease the score by one point.

4. Running the Lab environment

You can use your editor of choice (e.g. PyCharm) and follow the instructions below. Note, the lab code requires Python version lower than 3.10 and depends on three additional modules: numpy, tkinter and ipython.

1. Console: to run the code execute the following command in the Lab2 directory:

   python3 run_lab2.py

2. PyCharm: setup and start the PyCharm editor:

   - To add the PyCharm editor to your environment execute the following command in the terminal: module add prog/pycharm-professional/2021.2.3 or alternatively module initadd prog/pycharm-professional/2021.2.3 to always add it when logging in - note that you will need to open a new terminal window for the command to take effect the first time.
   - pycharm.sh
   - After starting the PyCharm editor, open the project by selecting File/Open and choose the Lab2 directory.
   - To execute run run_lab2.py file - open the file in the editor, right click and choose Run run_lab2.

5. Detailed Lab Task Description

Play around with the GUI and try the different options and agents.

1. Using myvacuumagent.py as a template, implement an agent with breadth-first search (BFS) algorithm.

   Note: The agent starts from a random location, which is achieved by executing the method in the move_to_random_start_position in Python. You should not change this part of the code. In Python, simply edit myvacuumagent.py file and run the lab again. Evaluate the performance of the agent.

2. In this task, you will extend and improve your agent to be able to solve the problem with depth first search (DFS) algorithm.

   Note: If you are already familiar with search algorithms, feel free to implement an agent based on search which shuts down at the home position after all the dirt is cleaned. In that case, the iterationCounter should be set to a sufficiently large value so that the agent can finish the task.

3. Write explanations of how your BFS and DFS works (in both Task 1 and 2) as follow:

	Breadth-First Search			Depth-First Search		
Environment size (without obstacles)	**#nodes explored**	**Solution length**	**Is it optimal? Score?**	**#nodes explored**	**Solution length**	**Is it optimal? Score?**
5x5						
10 x10						
15x15						
20x20						

What to submit:

Your submission should include the following:

1. Demonstrate your task 1 and 2 to the lab instructor.
2. Lab report answers from Task 3 and source code + README (how to compile and run your code).
3. Please create a folder called "yourname_studentID_Lab2" that includes all the required files and generate a zip file called "yourname_studentID_Lab2.zip".
4. Please submit your work (.zip) to Blackboard.